

Integração Contínua II

Semana 5

CSDV-246 - DevOps



Conteúdo

- Análise de código estático
- SonarQube
- Perfis de qualidade
- *Quality Gates*
- Métricas de código
- Integração de CI
- Notificações e relatórios



Análise de código estático

Análise de código estático



- Também conhecido como "análise estática de programa", "varredura de código" ou "linting".
- É a prática de analisar o **código sem executá-lo** e identificar possíveis bugs, vulnerabilidades de segurança e problemas de estilo ou arquitetura.
- Essa análise é realizada com ferramentas especializadas que integram os padrões de qualidade e segurança definidos pela equipe de desenvolvimento.
- Ele detecta vários tipos de problemas, como sintaxe não analisável, módulos não importados, código inacessível e não utilizado, código duplicado, código potencialmente vulnerável, violações de padrões e convenções de codificação.

Análise de código estático



- A detecção precoce de problemas de código no ciclo de vida de desenvolvimento de software (SDLC) leva a um software mais eficiente e confiável.
- A seguir estão os principais motivos para realizar a análise de código estático:
 - Detectar erros antes que eles afetem a produção. Os erros encontrados no estágio inicial do SDLC são muito mais baratos de corrigir.
 - Revelar vulnerabilidades de segurança e evite que elas explorem o ambiente de produção.

Análise de código estático



- Melhorar a capacidade de manutenção eliminando código duplicado e não utilizado.
- Alinhar o estilo do código entre os membros da equipe, fazendo com que a revisão de código se concentre mais na intenção do que nas discrepâncias estéticas.
- Atualmente, as ferramentas de análise de código estático analisam o código primeiro decompondo-o em uma Árvore de Sintaxe Abstrata (AST).

Análise de código estático



- Uma AST é uma estrutura de dados que divide o código, incluindo variáveis, funções e estruturas de controle, em partes menores e organiza essas partes em uma estrutura semelhante a uma árvore que é mais fácil de entender e analisar.
- Usando uma AST, os analisadores estáticos podem se concentrar em problemas lógicos sem se preocupar com as especificidades da linguagem de programação.

Análise de código estático



- Os analisadores de código estático de hoje seguem estes princípios fundamentais:
 1. **A análise de código estático** significa que esses analisadores não compilam ou executam o código para testá-lo. Em vez disso, eles criam ASTs e os usam para encontrar código não utilizado ou código que pode levar a erros se entradas específicas forem fornecidas.
 2. **As regras predefinidas** são armazenadas em uma base de dados de conhecimento. Essas regras estão relacionadas a padrões de codificação, práticas recomendadas, vulnerabilidades de segurança e complexidade de código.

Análise de código estático



- O analisador verifica se o código dentro do AST está em conformidade com essas regras. Por exemplo, um analisador pode observar como uma variável é declarada e acessada no AST e detectar se ela está sendo lida antes de definir seu valor.
- Muitos analisadores estáticos permitem que os desenvolvedores configurem quais regras aplicar e seu nível de gravidade.

Análise de código estático



- Os analisadores de código podem identificar falsos positivos no código (ou seja, relatar defeitos que não são problemas reais).
- A equipe de desenvolvimento deve revisar os resultados e identificar e gerenciar cada falso positivo adequadamente.
- Da mesma forma, é fundamental que os desenvolvedores revisem o código em busca de possíveis problemas de manutenção e arquitetura de código de alto nível que podem ser perdidos pelos analisadores.

Análise de código estático



- Por exemplo:
 1. Um engenheiro da equipe pode identificar se um padrão de design, como o padrão Factory, está sendo usado em excesso ou de forma inadequada em uma base de código.
 2. Eles também podem detectar casos em que uma abstração é inadequada, levando a um código complicado que é difícil de entender e manter.

Análise de código estático



Escolhendo o analisador de código estático adequado

- **Certifique-se de que o analisador seja compatível com suas linguagens de programação:** Alguns analisadores suportam uma ampla variedade de linguagens, enquanto outros se concentram em um em particular.
- **Considere os recursos do analisador:** Alguns analisadores detectam problemas de estilo de código, enquanto outros podem detectar vulnerabilidades de segurança e possíveis otimizações de desempenho.
- Embora os analisadores completos sejam preferidos, eles têm um custo adicional e podem exigir mais esforço para serem configurados.

Análise de código estático



Escolhendo o analisador de código estático adequado

- Analisadores limitados, gratuitos ou baratos geralmente permitem que você ative ou desative regras, enquanto analisadores mais avançados permitem que você especifique a gravidade de diferentes problemas e até mesmo crie regras personalizadas.
- Considere a integração com pipelines de CI/CD, isso é vital para incorporar a análise automatizada de código estático no fluxo de trabalho de desenvolvimento de software.
- Alguns analisadores também podem ser executados nos computadores dos desenvolvedores e integrar-se diretamente aos IDEs.

Análise de código estático



Opções do analisadores



Sonar

Checkmarx



ESLint

SonarQube

SonarQube



Sonar Products

sonarqube 

Solução no local

sonarcloud 

Solução de software como serviço

sonarlint 

Plugin para IDEs



- É uma ferramenta de análise local projetada para detectar problemas de código em mais de 30 linguagens, estruturas e plataformas de infraestrutura como código.
- Possui integração direta com pipelines de CI.
- O código é verificado em relação a um grande conjunto de regras que abrangem vários atributos de código, como capacidade de manutenção, confiabilidade e problemas de segurança em cada Merge/Pull Request.

Enfoque Clean Code

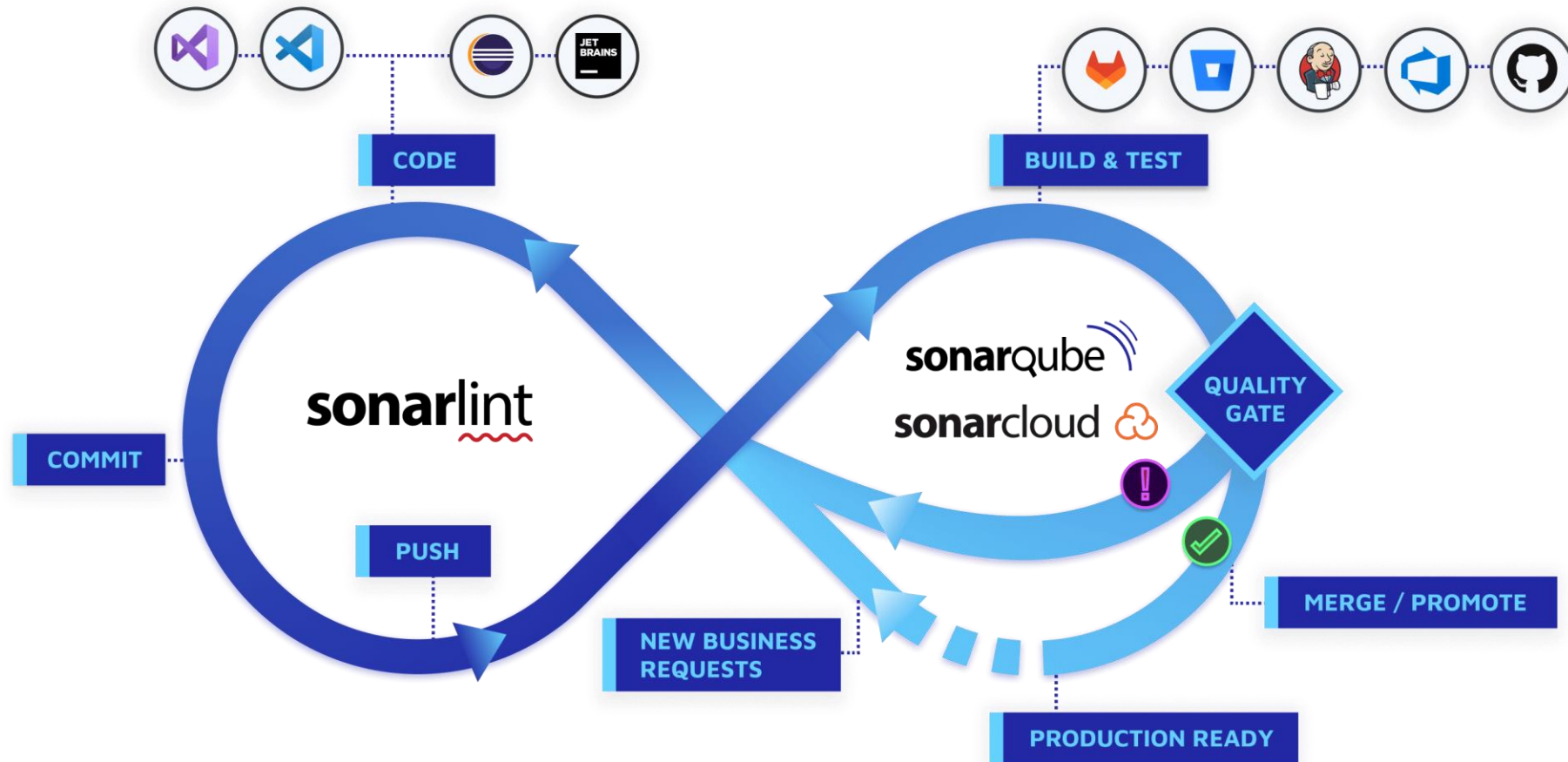
É o padrão para todo código que resulta em software seguro, confiável e sustentável. Escrever código limpo é essencial para manter uma base de código saudável.

- Limpeza contínua do código: garante que o novo código atenda aos altos padrões de segurança, confiabilidade e facilidade de manutenção.
- Perfis de qualidade: Use perfis predefinidos, como "Sonar Way", para aplicar regras de qualidade específicas da linguagem.
- Melhoria incremental: A aplicação contínua de padrões de código limpo melhora progressivamente toda a base de código.

SonarQube



Enfoque Clean Code





Como funciona?

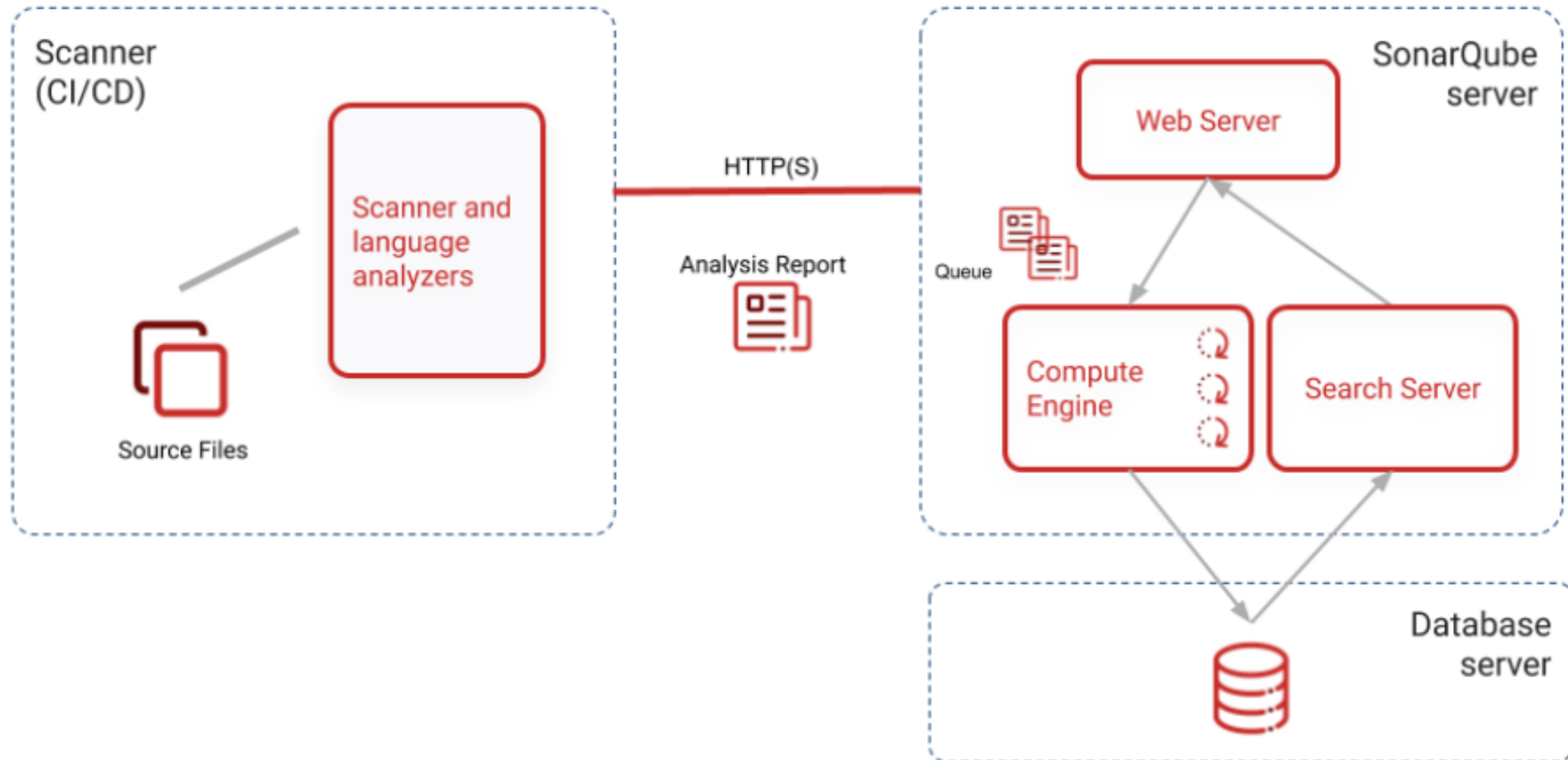
A instância do SonarQube compreende três componentes:

- **Servidor:** onde os seguintes processos são executados:
 - Servidor Web: Serve a interface do usuário.
 - Compute Engine: processa relatórios de análise de código e os salva no banco de dados.
- **Base de dados:** Armazene as seguintes informações:
 - Qualidade de código e métricas de segurança e problemas gerados.
 - A configuração do SonarQube.
- **Scanner:** Isso executa a análise de código no ambiente de desenvolvimento ou no pipeline de CI.

SonarQube



Como funciona?





SonarScanner

- O SonarScanner realiza a análise do código-fonte.
- Este é um programa autônomo que pode ser baixado e executado na estação de trabalho de cada desenvolvedor. Ele também pode ser executado no pipeline de CI.
- Envia os resultados da análise para o servidor SonarQube, onde os calcula, calcula os resultados dos cálculos realizados.
- Para realizar a análise, este scanner usa os analisadores de linguagem que ele baixa do servidor SonarQube.

Processo de análise com o SonarScanner

- O SonarScanner verifica o repositório local e determina os arquivos a serem verificados de acordo com o escopo de verificação configurado.
- O verificador envia uma solicitação de verificação ao respectivo analisador de idioma, que recupera os arquivos a serem verificados do sistema de arquivos e os analisa de acordo com o perfil de qualidade configurado.
- O analisador envia os resultados da análise (medições de qualidade e incidentes) para o scanner, que os envia para o servidor SonarQube na forma de um relatório.

Processo de análise com o SonarScanner

- O servidor SonarQube calcula os resultados da análise de forma assíncrona para executar o seguinte:
 - Ele identifica novos problemas de acordo com a nova definição de código configurada e os gera no novo código e no código geral.
 - Calcule o Quality Gate.
 - Gerar relatórios.



SonarScanner CLI

- Esse é o verificador usado quando não há um verificador específico para o sistema de compilação usado pela equipe de desenvolvimento.
- Scanners específicos estão disponíveis para as seguintes tecnologias:
 - [SonarScanner para Maven](#).
 - [SonarScanner para Gradle](#).
 - [SonarScanner para NPM](#).
 - [SonarScanner para .NET](#).

Configuração de análise de projeto

- Criar um projeto no SonarQube
 - O repositório de código-fonte é representado no SonarQube como um projeto.
 - O projeto pode ser criado usando a GUI do SonarQube antes de executar a primeira verificação ou pode ser criado automaticamente quando a verificação é iniciada pela primeira vez a partir de um SonarScanner.
- A análise realizada pelo SonarScanner é configurada através de parâmetros de análise.
 - Alguns parâmetros de análise são obrigatórios e outros têm um valor padrão, mas podem ser ajustados conforme necessário.

Configuração de análise de projeto

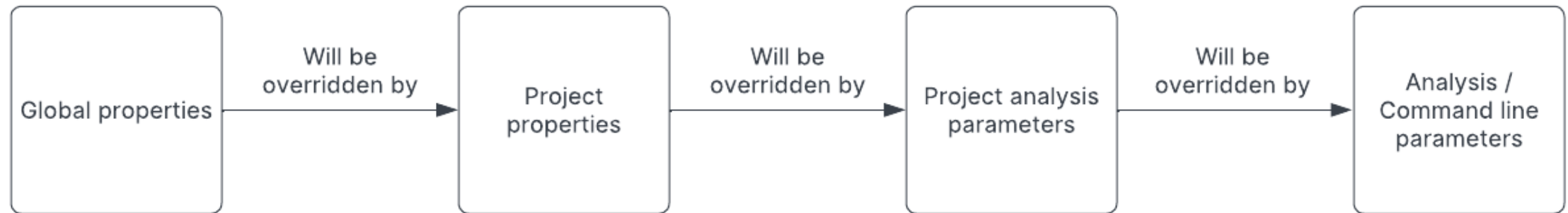
- O SonarQube gerencia os parâmetros de análise por meio das propriedades do sonar, que possui a seguinte sintaxe:
 - `sonar.<propiedade>`
- Os parâmetros de análise permitem que o código e a cobertura de teste sejam incluídos na análise.

Parâmetros de análise

- As configurações de análise do projeto podem ser configuradas em vários lugares
- Cada plug-in e analisador de idioma adiciona suas próprias "propriedades" que podem ser definidas na interface do usuário do SonarQube.
- Essas propriedades também podem ser definidas como parâmetros de análise.
- Abaixo está a hierarquia de propriedades em ordem de proveniência:

Parâmetros de análise

Settings Hierarchy



Parâmetros de análise

1. **Propriedades globais:** Aplica-se a todos os projetos. Definido na interface do usuário em Configurações do > Administração > Configurações gerais
2. **Propriedades do projeto:** Aplicar a um único projeto. No nível do projeto, definido na interface do usuário em Configurações do Projeto > Configurações Gerais
3. **Parâmetros de análise do projeto:** são definidos em um arquivo de configuração de análise de projeto ou em um arquivo de configuração de scanner
4. **Parâmetros de análise/linha de comando:** Eles são definidos quando você inicia uma varredura com na linha de comando -D

Parâmetros de análise

- **Parâmetros necessários:**

Propriedade	Descrição
<code>sonar.token</code>	Token usado pelo scanner para autenticar no servidor SonarQube.
<code>sonar.host.url</code>	A URL do seu servidor SonarQube. Exemplo <code>http://localhost:9000</code>
<code>sonar.projectKey</code>	A chave única para o projeto. Pode incluir até 400 caracteres.

Parâmetros de análise

- Alguns parâmetros opcionais (lista completa aqui):

Propriedade	Descrição
<code>sonar.projectName</code>	O nome do projeto que será exibido na interface do usuário do SonarQube.
<code>sonar.sources</code>	A análise de linha de base para o código-fonte principal (código não de teste) no projeto.
<code>sonar.projectBaseDir</code>	O diretório base do projeto. Use essa propriedade quando precisar que a verificação seja executada em um diretório diferente daquele a partir do qual ela foi iniciada.

Exemplos com a CLI do SonarScanner

- Especificar propriedades diretamente na linha de comando:

```
sonar-scanner -Dsonar.projectKey=myproject -Dsonar.sources=src1
```

- Digitalizando usando a imagem do Docker da CLI do SonarScanner:

```
docker run --rm -e SONAR_HOST_URL="http://${SONARQUBE_URL}" -e  
SONAR_LOGIN="myAuthenticationToken" -v "${YOUR_REPO}:/usr/src" -v  
${YOUR_CACHE_DIR}:/opt/sonar-scanner/.sonar/cache sonarsource/sonar-  
scanner-cli
```

- Você pode criar um arquivo de configuração para não especificar propriedades na linha de comando.

Arquivo "sonar-project.properties"

```
# must be unique in a given SonarQube instance
sonar.projectKey=my:project

# --- optional properties ---

# defaults to project key
#sonar.projectName=My project
# defaults to 'not provided'
#sonar.projectVersion=1.0

# Path is relative to the sonar-project.properties file. Defaults to .
#sonar.sources=.

# Encoding of the source code. Default is default system encoding
#sonar.sourceEncoding=UTF-8
```

Exemplos com a CLI do SonarScanner

- Você pode usar a propriedade "project.settings" para especificar o caminho do arquivo de configuração do projeto, por exemplo

```
sonar-scanner -Dproject.settings=../sonar-project.properties -  
Dsonar.projectKey=myproject -Dsonar.sources=src1
```
- Leve em consideração que "project.settings" não é compatível com a propriedade "sonar.projectBaseDir"

Quality Profiles

Quality Profiles



- Eles definem o conjunto de regras que serão aplicadas durante a análise de código.
- Cada projeto tem um perfil de qualidade definido para cada linguagem suportado. Ao analisar um projeto, o SonarQube determina quais linguagens são usadas e usa o perfil de qualidade ativo para cada uma dessas linguagens nesse projeto específico.
- O SonarQube vem com um perfil de qualidade integrado definido para cada idioma suportado chamado **Sonar way**

Quality Profiles



- Cada projeto tem um perfil de qualidade definido para cada linguagem suportada.
- Ao analisar um projeto, o SonarQube determina quais idiomas são usados e usa o Perfil de Qualidade ativo para cada um desses idiomas naquele projeto específico.
- O SonarQube vem com um perfil de qualidade integrado definido para cada idioma suportado, chamado Sonar-way (marcado com o rótulo BUILT-IN na interface).

Quality Profiles

[Projects](#)[Issues](#)[Rules](#)[Quality Profiles](#)[Quality Gates](#)[More](#)

Quality Profiles

Quality profiles are collections of rules to apply during an analysis. For each language there is a default profile. All projects not explicitly assigned to some other profile will be analyzed with the default. Ideally, all projects will use the same profile for a language. [Learn More](#)

Filter by

Select language



AzureResourceManager, 1 profile(s)

Projects ?

Rules

Updated

Used

[Sonar way](#) BUILT-IN

DEFAULT

[31](#)

8 minutes ago

Never

C#, 1 profile(s)

Projects ?

Rules

Updated

Used

[Sonar way](#) BUILT-IN

DEFAULT

[335](#)

8 minutes ago

Never

CSS, 1 profile(s)

Projects ?

Rules

Updated

Used

[Sonar way](#) BUILT-IN

DEFAULT

[24](#)

8 minutes ago

Never

CloudFormation, 1 profile(s)

Projects ?

Rules

Updated

Used

Recently Added Rules

[Test class names should end with "Test"](#)

PHP, activated on 1 profile(s)

[All branches in a conditional structure should not have exactly the ...](#)

PHP, activated on 1 profile(s)

[A new session should be created during user authentication](#)

PHP, activated on 1 profile(s)

["session.use_trans_sid" should not be enabled](#)

PHP, not yet activated

[Variables should be initialized before use](#)

PHP, activated on 1 profile(s)

[Functions should use "return" consistently](#)

PHP, activated on 1 profile(s)

[String literals should not be concatenated](#)

PHP, not yet activated

[Encrypting data is security-sensitive](#)

PHP, not yet activated

[The output of functions that don't return anything should not be us...](#)

PHP, activated on 1 profile(s)



Quality Gates

Quality Gates



- A Quality Gates aplica uma política de qualidade que responde a esta pergunta: meu projeto está pronto para lançamento?
- Para responder a esta pergunta, é definido um conjunto de condições em relação às quais os projetos são medidos, por exemplo:
 - No new issues
 - Code Coverage on new code greater than 80%
- Idealmente, todos os projetos devem usar a mesma *quality gate*, mas isso nem sempre é prático. Você pode definir quantas *gates* de qualidade precisar.

Quality Gates



- Cada condição de *quality gate* é uma combinação de:
 - Uma medida
 - Um operador de comparação
 - Um valor de erro
- Por exemplo, uma condição pode ser:
 - Medida: Blocker issue
 - Operador de comparação: > (is greater than)
 - Error: 0
- De acordo com a definição deste Quality Gate pode ser expresso que: Não Há problemas de bloqueio no Código (se ele passar).

Quality Gates

[Projects](#)[Issues](#)[Rules](#)[Quality Profiles](#)[Quality Gates](#)[Administration](#)[More](#)

Quality Gates ?

Create

Sonar way

DEFAULT BUILT-IN



Sonar way

DEFAULT

BUILT-IN

Copy

The only quality gate you need to practice [Clean as You Code](#)

Conditions

Your new code will be clean if: ?

New code has 0 issues

All new security hotspots are reviewed

New code is sufficiently covered by test

Coverage is greater than or equal to 80.0% ?

New code has limited duplication

Duplicated Lines (%) is less than or equal to 3.0% ?

Projects ?

Every project not specifically associated to a quality gate will be associated to this one by default.

Quality Gates



Quality Gates ? [Create](#)

- myQG
- Sonar way
- DEFAULT BUILT-IN

Add Condition [X]

Where?

- ☒ On New Code
- ☐ On Overall Code

Quality Gate fails when

Blocker Issues

Operator is greater than Value

0

[Add Condition](#) [Close](#)

[Add Condition](#)

Value
0
100%
80.0%
3.0%

SonarQube™ technology is powered by [SonarSource SA](#)

Community Edition v10.7 (96327) ACTIVE LGPL v3 Community Documentation Plugins Web API

Quality Gates



Overview

Issues

Security Hotspots

Security Reports

Measures

Quality Gate Status ?



Quality Gate
Passed



Enjoy your sparkling clean code!

Métricas de Código

Métricas de Código



- O SonarQube inclui várias métricas agrupadas nas seguintes categorias, para ver a lista completa de métricas por categorias, confira [link](#):
 1. Complexidade
 2. Cobertura
 3. Duplicação
 4. Incidentes
 5. Manutenibilidade
 6. Confiabilidade
 7. Segurança
 8. Tamanho
 9. Quality Gates

Métricas de Código



Abaixo estão algumas métricas por categoria, apenas como exemplo:

- **Categoria:** Complexidade

Métrica	Clave de Métrica	Definição
Complexidade ciclomática	<code>complexity</code>	Uma métrica quantitativa usada para calcular o número de caminhos por meio do código. Veja abaixo.
Complexidade Cognitiva	<code>cognitive_complexity</code>	Uma classificação de como é difícil entender o fluxo do controle de código.

Métricas de Código



- **Categoria:** Cobertura

Métrica	Clave de Métrica	Definição
Cobertura	coverage	Uma combinação de cobertura de linha e cobertura de condição.
Testes unitários	tests	O número de testes unitários.
Densidade de sucesso do teste unitários(%)	test_success_density	$\text{unitTestSuccessDensity} = \frac{(\text{unitTests} - (\text{unitTestErrors} + \text{unitTestFailures}))}{(\text{unitTests})} * 100$

Métricas de Código



- **Categoria:** Security

Métrica	Clave de Métrica	Definição
Classificação de segurança	security_rating	Qualificação relacionada à segurança. A = 0 vulnerabilidade B = pelo menos uma vulnerabilidade menor C = pelo menos uma vulnerabilidade Importante D = pelo menos uma vulnerabilidade crítico E = pelo menos uma vulnerabilidade do bloqueador
Classificação de revisão de segurança	security_review_rating	A classificação de revisão de segurança é uma classificação de letra baseada na porcentagem de pontos de acesso de segurança revisados.

Integração com CI



Integração de CI

Exemplo em um pipeline

- Podemos automatizar a análise em nossos pipelines de CI.
- Para fazer isso, usaremos o SonarScanner do pipeline.

stages:

- lint
- static_analysis

lint_job:

stage: lint

script:

- eslint .

static_analysis_job:

stage: lint

script:

- sonar-scanner

Integração de CI



Etapas de integração

- Etapa 1: um desenvolvedor envia as alterações para uma ramificação no repositório remoto.
- Etapa 2: um pipeline de CI é executado para a ramificação específica.
- Etapa 3: o pipeline clona o repositório remoto e extrai a ramificação correspondente para o repositório local no host de CI/CD.
- Passo 4: No caso de uma linguagem de programação compilada, o pipeline compila o código.
- Etapa 5: o pipeline executa o scanner de sonar apropriado para analisar o código.
- Etapa 6: O scanner envia os resultados do teste para o servidor SonarQube, que os calcula.

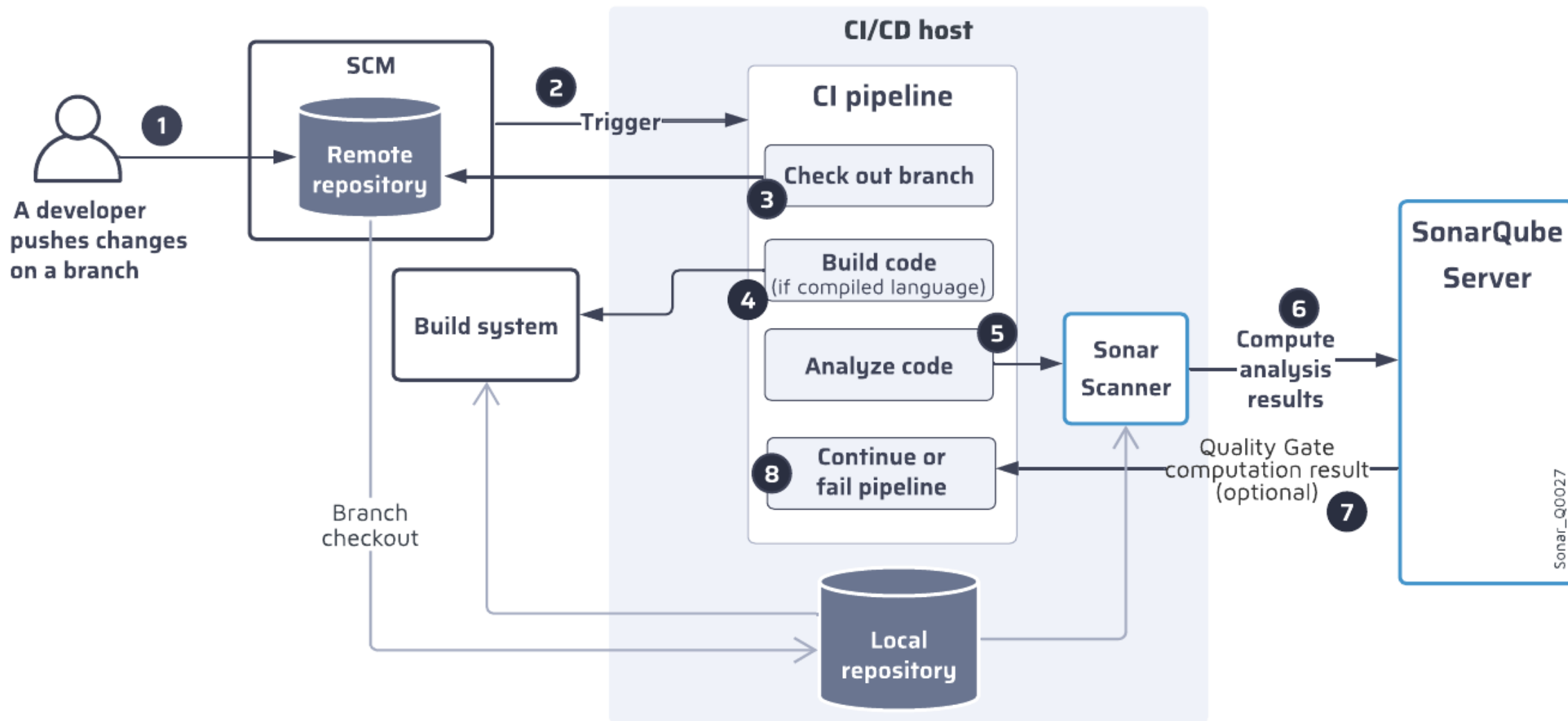
Integração de CI



Etapas de integração

- Etapa 7: O servidor envia o resultado do cálculo do Quality Gate para o pipeline de CI (esta etapa é opcional).
- Etapa 8: O pipeline continua (se o Quality Gate for bem-sucedido) ou para (se não).

Integração de CI



Exemplo de análise estática com Java Maven

```
sonarqube_scan:
```

```
  stage: static_analysis
```

```
  script:
```

```
    - mvn sonar:sonar -Dsonar.projectKey=$SONAR_PROJECT_KEY -  
      Dsonar.host.url=$SONAR_HOST_URL -Dsonar.login=$SONAR_TOKEN
```

```
quality_gate_check:
```

```
  stage: quality_gate
```

```
  script:
```

```
    - curl -X GET  
      "$SONAR_HOST_URL/api/qualitygates/project_status?projectKey=$SONAR_PROJECT_KEY"  
    -u "$SONAR_TOKEN:" | jq -r '.projectStatus.status' | grep -q "OK"
```

Exemplo de análise estática com NodeJS

```
sonarqube_scan:
```

```
  stage: static_analysis
```

```
  script:
```

```
    - sonar-scanner -Dsonar.projectKey=$SONAR_PROJECT_KEY -Dsonar.sources=. -  
      Dsonar.host.url=$SONAR_HOST_URL -Dsonar.login=$SONAR_TOKEN
```

```
quality_gate_check:
```

```
  stage: quality_gate
```

```
  script:
```

```
    - curl -X GET  
      "$SONAR_HOST_URL/api/qualitygates/project_status?projectKey=$SONAR_PROJECT_KEY"  
      -u "$SONAR_TOKEN:" | jq -r '.projectStatus.status' | grep -q "OK"
```

Exemplo de análise estática com DotNet

sonarqube_scan:

stage: static_analysis

script:

- dotnet sonarscanner begin /k:"\$SONAR_PROJECT_KEY"
/d:sonar.host.url="\$SONAR_HOST_URL" /d:sonar.login="\$SONAR_TOKEN"
- dotnet build
- dotnet sonarscanner end /d:sonar.login="\$SONAR_TOKEN"

quality_gate_check:

stage: quality_gate

script:

- curl -X GET
"\$SONAR_HOST_URL/api/qualitygates/project_status?projectKey=\$SONAR_PROJECT_KEY"
-u "\$SONAR_TOKEN:" | jq -r '.projectStatus.status' | grep -q "OK"

Notificações e relatórios

Notificações e relatórios



- As notificações permitem que os desenvolvedores resolvam os bugs imediatamente, ajudando a manter a eficiência.
- Os relatórios fornecem informações detalhadas sobre qualidade de código, métricas de desempenho e conformidade, permitindo monitorar tendências, gerenciar dívidas técnicas e tomar decisões informadas.
- Ambos promovem transparência, responsabilidade e melhoria contínua, que são fundamentais para o sucesso no DevOps

Notificações e relatórios



- Por padrão, as plataformas de DevOps na nuvem, como o GitLab CI, enviam notificações por e-mail quando uma compilação de pipeline é concluída, informando se ela foi bem-sucedida ou falhou e nos fornecendo um link para a compilação específica para visualizar os logs do pipeline.
- Essas notificações podem ser gerenciadas de acordo com a documentação fornecida pela plataforma DevOps, ou a ferramenta CI, no caso do GitLab, confira este [link](#).
- Além disso, o SonarQube também permite que as notificações sejam enviadas por e-mail.

Notificaciones y Reportes



Administration

ConfigurationSecurityProjectsSystemMarketplace

General Settings

Edit global settings for this SonarQube instance.

Find in Settings

Analysis Scope

Authentication

DevOps Platform Integrations

Email Notification

External Analyzers

General

Housekeeping

JaCoCo

Languages

SMTP Configuration

Follow the steps below to configure and test your authentication.

1 Select your authentication type and complete the fields below



Basic Authentication

Authenticate with a username and password



Modern Authentication

Authenticate with OAuth
Supported providers: Microsoft



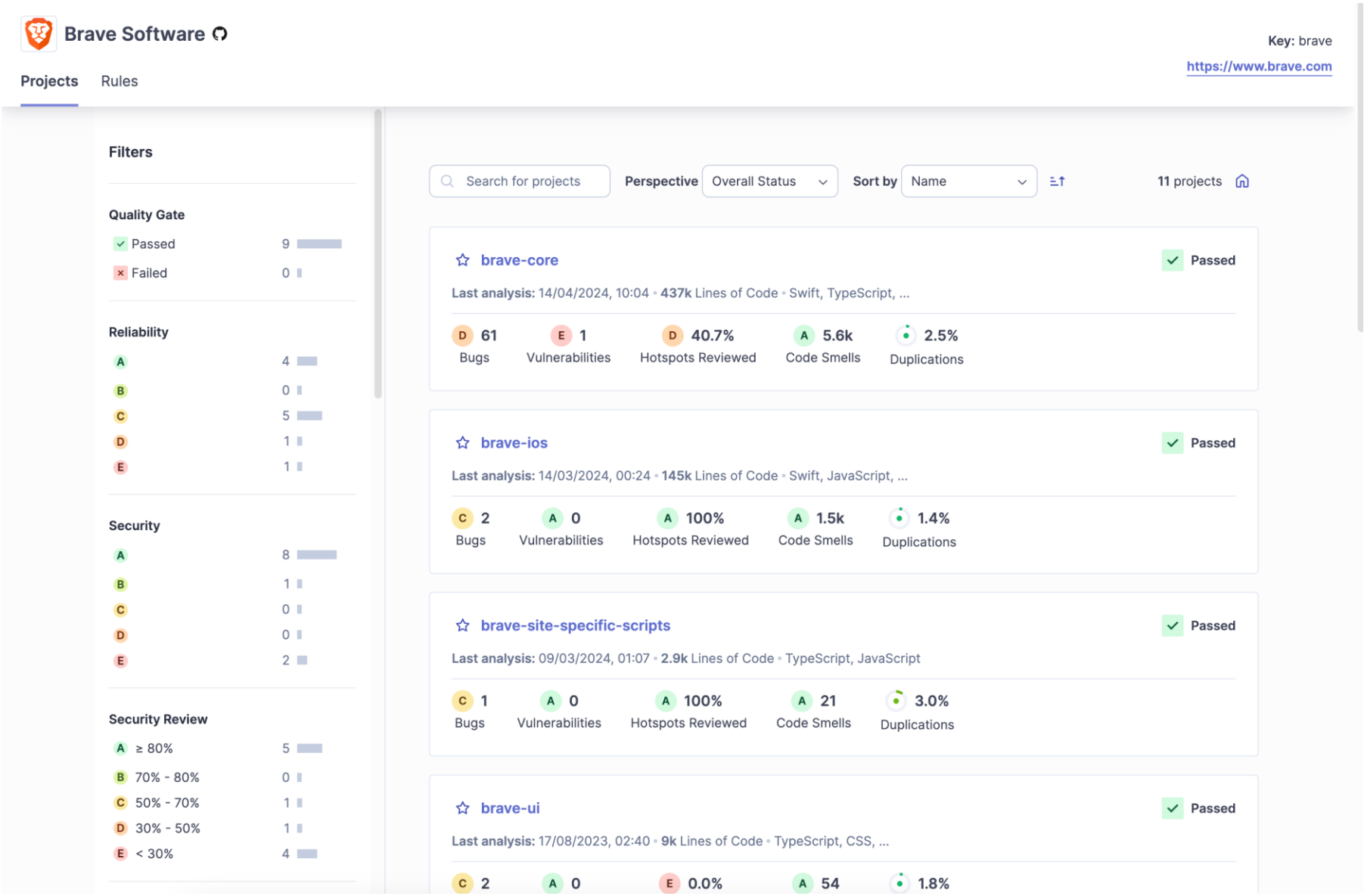
Recommended for stronger security compliance

SMTP username *

Username used to authenticate to the SMTP

server

Notificaciones y Reportes



Q&A